Pandas Quick Reference

A practical cheat sheet for remembering methods after class

How to use this sheet

When you forget syntax, find the task you want to do and copy the pattern. Replace df, column names, and values with your own. You do not need to memorize every line—remember what each method does.

1. Import Pandas and create data

Import Pandas with the usual alias:

```
import pandas as pd
```

Create a Series (one-dimensional labeled data):

```
s = pd.Series([10, 20, 30, 40, 50])
print(s)
print(s.index)   # index labels
print(s.dtype)   # data type of values
```

Create a DataFrame from a list/tuple of rows:

```
data = (("Akash", 22, "Dhaka"),
        ("Raihan", 21, "Barishal"),
        ("Habib", 18, "Sylhet"),
        ("Ash", 20, "Pallet Town"))
df = pd.DataFrame(data, columns=["Name", "Age", "City"])
```

Create a DataFrame from a dictionary of columns:

```
data = {"Name": ["Akash", "Raihan"], "Age": [22, 21]}
df = pd.DataFrame(data)
```

2. Load data from files

```
df = pd.read_csv("file.csv")
df = pd.read_excel("file.xlsx")
df = pd.read_parquet("file.parquet")
df = pd.read_json("file.json")
```

The required optional reader may need an extra library installed (for example, Excel support).

3. Inspect a DataFrame

```
df.head()          # first 5 rows by default
df.head(4)         # first 4 rows
df.tail()          # last 5 rows by default
df.sample(3)       # 3 random rows
df.columns         # column labels
df.index           # row index labels
df.index.tolist()  # index labels as a Python list
df.info()          # column types, non-null counts, memory info
df.describe()      # summary statistics for numeric columns
```

```
type(df)           # Python object type
df.dtypes          # data type of each DataFrame column
```

4. Select columns and rows

```
df['FullName']                          # one column (Series)
df[['FullName', 'Python Marks']]        # multiple columns (DataFrame)
df.loc[0]                               # row with index label 0
df.loc[[2, 3, 19]]                      # selected index labels
df.loc[3:7]                             # labels 3 through 7 (inclusive)
df.loc[:, 'Python Marks']               # all rows, one column
df.loc[:, ['Python Marks', 'Algorithm Marks']]  # all rows, selected columns
df.loc[3:7, 'CompletionStatus']          # selected rows and one column
df.iloc[0]                              # first row by position
df.iloc[:, 0:5]                         # all rows, first 5 columns
```

Key difference: .loc uses index/column labels; .iloc uses integer positions. With .loc, a label slice such as 3:7 includes label 7. With .iloc, 0:5 stops before position 5.

5. Filter rows with conditions

```
df.loc[df['CompletionStatus'] == 'Completed']
df.loc[df['Data Structure Marks'] >= 90]
df.loc[(df['CompletionStatus'] == 'Completed') &
       (df['Data Structure Marks'] >= 90)]
df.loc[(df['Age'] >= 18) | (df['City'] == 'Dhaka')]
```

Use & for element-wise AND and | for element-wise OR when combining Pandas column conditions. Put each condition in parentheses. Do not use Python's and/or for Pandas Series conditions.

Missing values: NaN means a value is missing. To find missing values, use df['Column'].isna(); to find non-missing values, use df['Column'].notna().

6. Sort rows

```
df.sort_values('Data Structure Marks')  # ascending by default
df.sort_values('Data Structure Marks', ascending=False)  # descending
df.sort_values(['Data Structure Marks', 'Python Marks'],
               ascending=[False, True], na_position='last')
```

For multiple columns, each item in ascending matches the corresponding column. False = descending; True = ascending. Sorting returns a new DataFrame unless you assign it or use an in-place option where supported.

7. Change index, columns, and values

```
df2 = df.set_index('StudentID')
df.rename(columns={'FullName': 'Full Name'}, inplace=True)
df.loc[1, 'Python Marks'] = 90
df.loc[1, 'CompletionStatus'] = 'Completed'
df.loc[1:3, 'Python Marks'] += 2
```

set_index() returns a new DataFrame unless inplace=True is used. Be careful with inplace=True because it changes the original DataFrame.

8. Delete rows or columns

```
df.drop(0)                    # return copy without index label 0
df.drop('Instructor', axis=1)  # return copy without this column
df.drop(0, inplace=True)       # remove row from original DataFrame
df.drop('Instructor', axis=1, inplace=True)  # remove column
```

axis=0 means rows; axis=1 means columns. Use inplace=True only when you want to modify the original DataFrame.

9. Iterate through rows

```
for index, row in df.iterrows():
    print(index, row)

for row in df.itertuples(index=False):
    print(row)
```

Use iteration only when needed; many Pandas tasks can be done more simply with selection, filtering, or vectorized operations.

10. Common memory rules

- Series = one-dimensional; DataFrame = table with rows and columns.
- df['Column'] selects one column; df[['A', 'B']] selects multiple columns.
- df.index gives row labels; df.columns gives column labels.
- df.dtypes gives each column's data type; a Series uses .dtype.
- .loc = labels; .iloc = positions.
- Pandas multiple conditions: use (condition1) & (condition2), or (condition1) | (condition2).
- sort_values() sorts; ascending=False means largest to smallest.
- axis=0 means rows; axis=1 means columns.

Practice habit

Before checking this sheet, try to write the method from memory. If you forget, look it up, run it, and then close the sheet and write it once more. Understanding the goal and knowing how to find syntax is more important than memorizing every method.